



Update on the CPython JIT 🔥

What to expect in 3.15

Diego Russo
2026-07-15

Who am I

Diego Russo



diego@python.org

CPython Core Developer

- CPython JIT
- Runtime performance
- Compiler & interpreter internals

Principal Software Engineer

- Leading CPython work @ Arm
- Arm ecosystem enablement

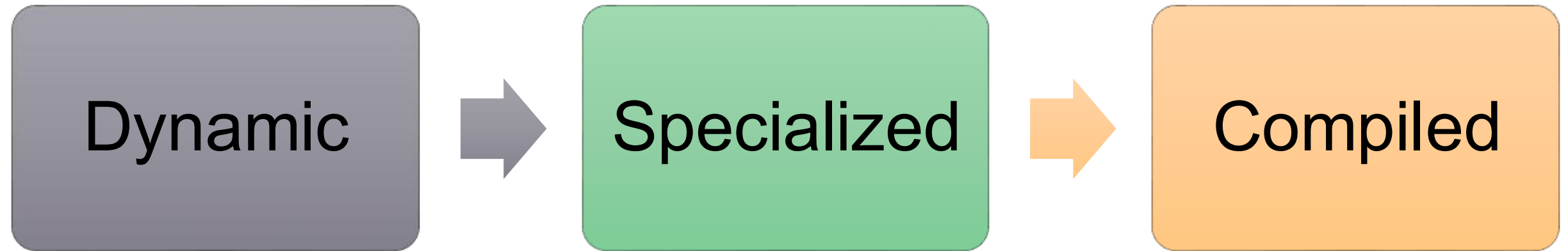
Community & Conferences

- EuroPython organizer & speaker
- PyCon Italia keynote speaker
- PyCon UK speaker

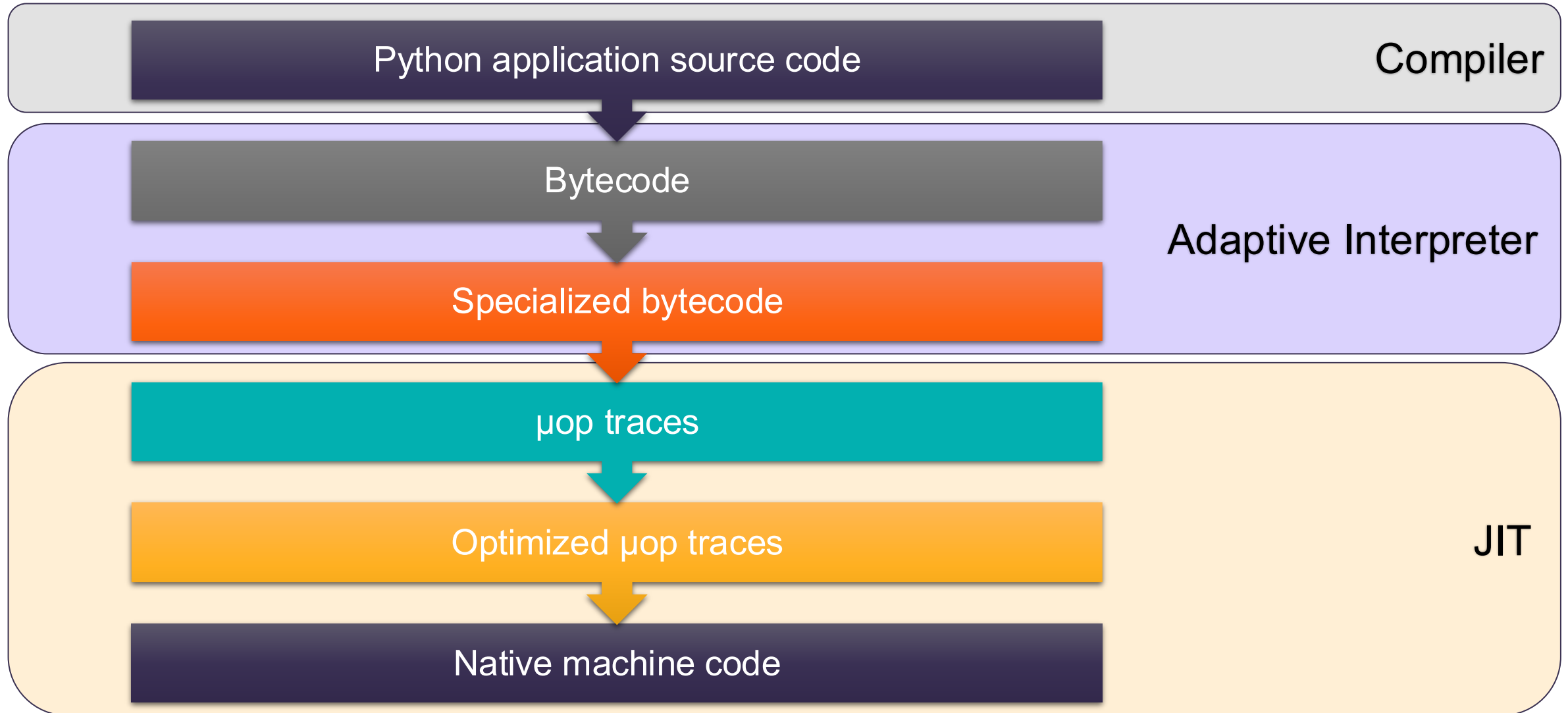
From explanation to expectation 🧭

Recap

From adaptive to compiled execution



How CPython executes and optimizes your code



Example: hot loop

Python source

```
def b():  
    total = 0  
    for i in range(10000):  
        total += i
```

Python Bytecode

```
1  RESUME_CHECK 0 0  
2  LOAD_SMALL_INT 0  
   STORE_FAST 0 (total)  
3  LOAD_GLOBAL 1 (range + NULL)  
   LOAD_CONST 1 (10000)  
   CALL 1  
   GET_ITER 0  
L1: FOR_ITER_RANGE 11 (to L2)  
   STORE_FAST 1 (i)  
4  LOAD_FAST_BORROW_LOAD_FAST_BORROW 1 (total, i)  
   BINARY_OP_ADD_INPLACE 13 (+=)  
   STORE_FAST 0 (total)  
   JUMP_BACKWARD_NO_INTERRUPT 13 (to L1)  
3 L2: END_FOR  
   POP_ITER  
   LOAD_CONST 2 (None)  
   RETURN_VALUE
```

Example: hot loop

Python source

```
def b():  
    total = 0  
    for i in range(10000):  
        total += i
```

Python Specialized Bytecode

```
1  RESUME_CHECK      0  
2  LOAD_SMALL_INT    0  
   STORE_FAST        0 (total)  
3  LOAD_GLOBAL        1 (range + NULL)  
   LOAD_CONST         1 (10000)  
   CALL               1  
   GET_ITER           0  
L1: FOR_ITER_RANGE    11 (to L2)  
   STORE_FAST         1 (i)  
4  LOAD_FAST_BORROW_LOAD_FAST_BORROW 1 (total, i)  
   BINARY_OP_ADD_INT  13 (+=)  
   STORE_FAST         0 (total)  
   JUMP_BACKWARD_NO_JIT 13 (to L1)  
3 L2: END_FOR  
   POP_ITER  
   LOAD_CONST         2 (None)  
   RETURN_VALUE
```


Example: hot loop

μop trace

FOR_ITER_RANGE	BINARY_OP_ADD_INT
_CHECK_VALIDITY	_CHECK_VALIDITY
_SET_IP	_SET_IP
_ITER_CHECK_RANGE	_GUARD_TOS_INT
_GUARD_NOT_EXHAUSTED_RANGE	_GUARD_NOS_INT
_ITER_NEXT_RANGE	_BINARY_OP_ADD_INT
STORE_FAST	_POP_TOP_INT
_CHECK_VALIDITY	_POP_TOP_INT
_SET_IP	STORE_FAST
_SWAP_FAST	_CHECK_VALIDITY
_POP_TOP	_SET_IP
LOAD_FAST_BORROW_LOAD_FAST_BORROW	_SWAP_FAST
_CHECK_VALIDITY	_POP_TOP
_SET_IP	_JUMP_TO_TOP
_LOAD_FAST_BORROW	JUMP_BACKWARD
_LOAD_FAST_BORROW	_CHECK_VALIDITY
	_SET_IP
	_CHECK_PERIODIC

μop optimized trace

```

_CHECK_VALIDITY_r00
_ITER_CHECK_RANGE_r02
_GUARD_NOT_EXHAUSTED_RANGE_r22
_ITER_NEXT_RANGE_r23
_SET_IP_r33
_SWAP_FAST_1_r33
_SPILL_OR_RELOAD_r31
_POP_TOP_r10
_CHECK_VALIDITY_r00
_LOAD_FAST_BORROW_0_r01
_LOAD_FAST_BORROW_1_r12
_GUARD_TOS_OVERFLOWED_r22
_GUARD_NOS_INT_r22
_BINARY_OP_ADD_INT_r23
_POP_TOP_NOP_r32
_POP_TOP_NOP_r21
_SWAP_FAST_0_r11
_POP_TOP_INT_r10
_JUMP_TO_TOP_r00
    
```

Machine code

```

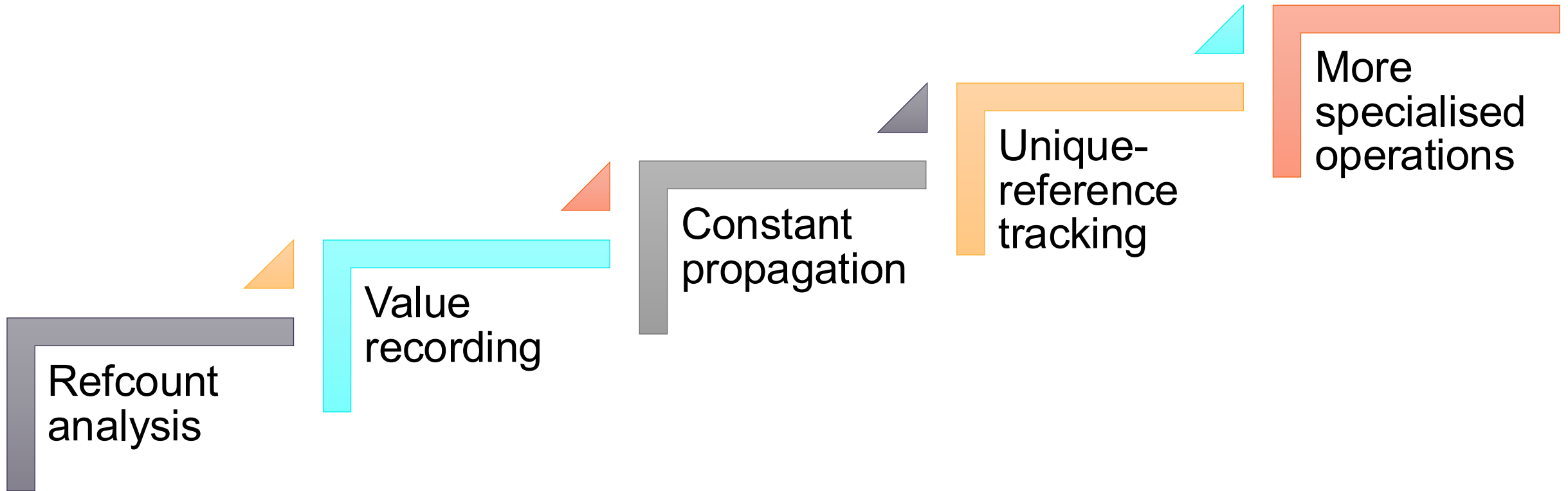
a8 82 5f f8 e9 03 28 2a 3f 05 40 f2 20 01 00 54 08 f9 7f 92 09 00 00 d0 08
05 40 f9 29 ed 46 f9 1f 01 09 eb 40 00 00 54 66 06 00 14 0d 00 00 14 fd 7b
bfa9 08 00 00 d0 00 00 00 d0 00 80 35 91 08 f1 46 f9 01 00 00 d0 21 f4 35
91 03 00 00 d0 63 80 36 91 22 3f 80 52 fd 03 00 91 00 01 3f d6 fd 7b bf a9
a8 82 5f f8 fd 03 00 91 e9 03 28 2a 3f 05 40 f2 c0 01 00 54 08 f9 7f 92 0a
00 00 d0 09 05 40 f9 4a 75 47 f9 3f 01 0a eb 41 02 00 54 08 11 40 f9 1f 01
00 f1 6d 00 00 54 fd 7b c1 a8 18 00 00 14 fd 7b c1 a8 0c 07 00 14 08 00 00
d0 00 00 00 d0 00 d4 39 91 08 79 47 f9 01 00 00 d0 21 48 3a 91 03 00 00
d0 63 d4 3a 91 22 3f 80 52 00 01 3f d6 08 00 00 d0 00 00 00 d0 00 a0 37
91 08 79 47 f9 01 00 00 d0 21 20 38 91 03 00 00 d0 63 94 38 91 42 0f 80
52 00 01 3f d6 1f 20 03 d5 fd 7b bf a9 a8 82 5f f8 fd 03 00 91 e9 03 28 2a
3f 05 40 f2 e0 06 00 54 08 f9 7f 92 0a 00 00 f0 09 05 40 f9 4a 79 40 f9 3f
01 0a eb 61 07 00 54 09 11 40 f9 3f 01 00 f1 4d 08 00 91 62 0f 80 52 00 01
3f d6 08 00 00 f0 00 00 00 d0 00 ec 3d 91 08 85 40 f9 01 00 00 d0 21 60 3c
91 03 00 00 d0 63 ac 3c 91 e2 0f 80 52 00 01 3f d6 08 00 00 f0 00 00 00 d0
00 18 3e 91 08 85 40 f9 01 00 00 d0 21 60 3c 91 03 00 00 d0 63 ac 3c 91
42 11 80 52 00 01 3f d6 08 00 00 f0 00 00 00 f0 00 dc 00 91 08 85 40 f9 01
00 00 d0 21 e4 3e 91 03 00 00 f0 63 3c 00 91 62 3b 80 52 00 01 3f d6 08
00 00 f0 00 00 00 f0 00 48 01 91 08 85 40 f9 01 00 00 f0 21 d8 01 91 03 00
00 f0 63 70 02 91 a2 02 80 52 00 01 3f d6 08 00 00 f0 00 00 00 f0 00 48 03
91
    
```

What changed in 3.15

LLVM 21 build-time dependency 🏗️

A better tracing frontend 🔍

More JIT optimisations



Top-of-stack caching

Basic register allocator

Without TOS caching

LOAD_FAST

local mem -> register -> stack mem

LOAD_FAST

local mem -> register -> stack mem

BINARY_OP_ADD_INT

stack mem -> register

stack mem -> register

register -> stack mem

With TOS caching

LOAD_FAST

local memory -> reg_0

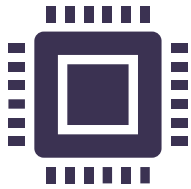
LOAD_FAST

local memory -> reg_1

BINARY_OP_ADD_INT

reg_0 + reg_1 -> reg_0

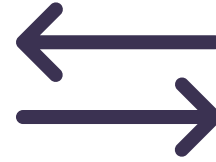
Better machine code generation



Lower generated-
code memory usage



More efficient
machine code



Better control-flow
shape

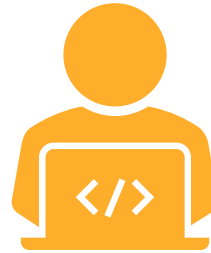


Fewer unnecessary
branches

Debugging and unwinding



GDB unwinding



GNU backtrace()
support



Better native stack
visibility

GDB example

Python 3.14: stack stops

```
#4 _PyEval_EvalFrame (tstate=<_PyRuntime+315696>, frame=<optimized out>,  
#5 _PyEval_Vector (tstate=<_PyRuntime+315696>, func=<optimized out>, loca  
#6 _PyObject_VectorcallTstate (tstate=<_PyRuntime+315696>, callable=, arg  
#7 PyObject_Vectorcall (callable=, args=<optimized out>, nargsf=<optimize  
#8 ?? ()  
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
```

Python 3.15: stack continues

```
#5 _PyEval_EvalFrame (tstate=<_PyRuntime+345120>, frame=<optimized out>  
#6 _PyEval_Vector (tstate=<_PyRuntime+345120>, func=<optimized out>, l  
#7 _PyObject_VectorcallTstate (tstate=<_PyRuntime+345120>, callable=,  
#8 PyObject_Vectorcall (callable=, args=<optimized out>, nargsf=<optim  
#9 _Py_BuiltinCallFastWithKeywords_StackRef (callable=..., arguments=<  
#10 py::jit:executor ()  
#11 _PyEval_EvalFrameDefault (tstate=, frame=, throwflag=-1428874900) a  
#12 _PyEval_EvalFrame (tstate=<_PyRuntime+345120>, frame=, throwflag=0)  
#13 _PyEval_Vector (args=, argcount=0, kwnames=, tstate=<_PyRuntime+3451
```

JIT + free-threading = not yet a working pair 🧵

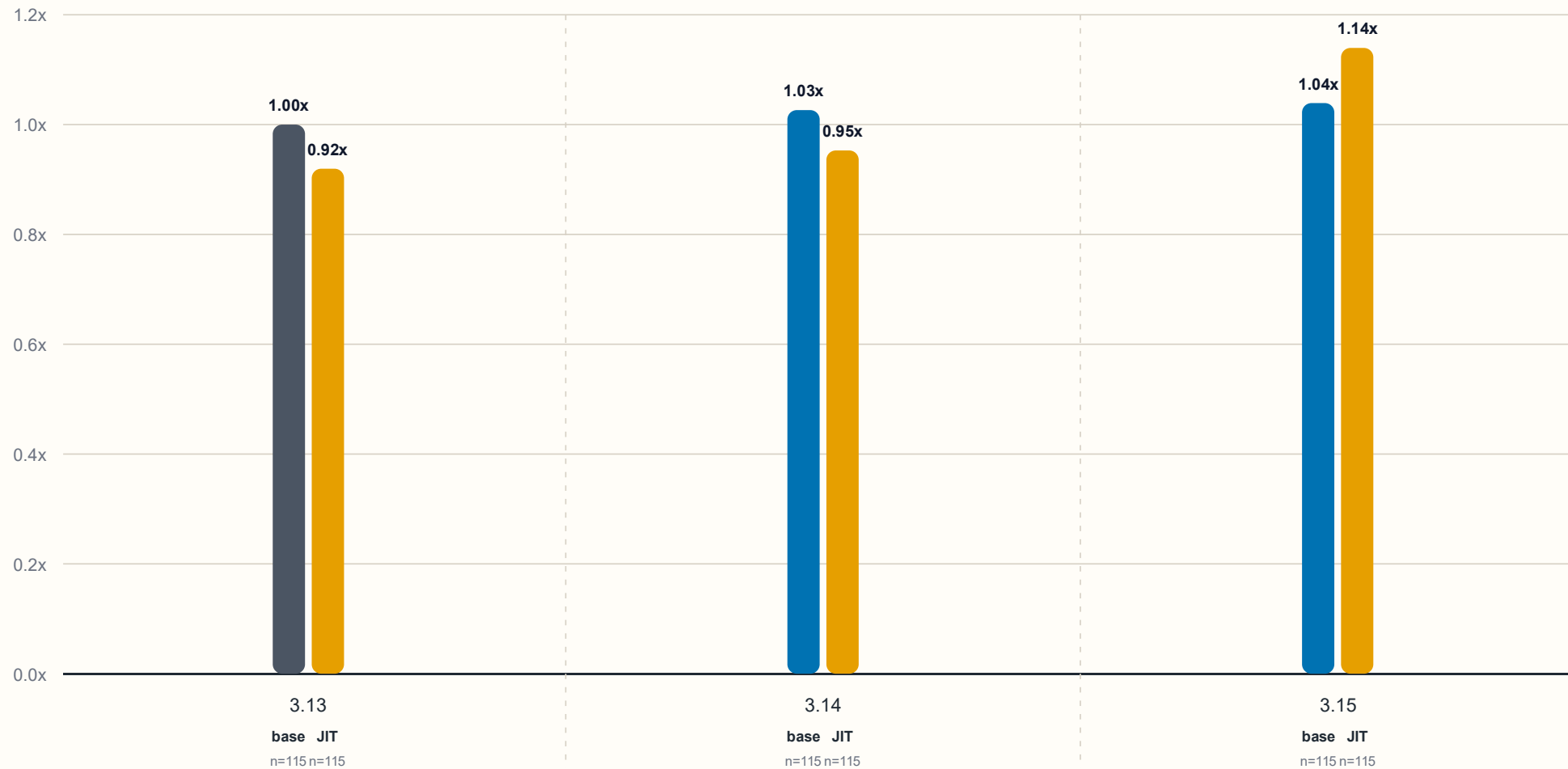
Performance

Why the JIT is not a silver bullet 🚀

CPython JIT focus: 3.13 to 3.15

Default interpreter and JIT builds only; geomean speedup vs CPython 3.13 default. Higher is better.

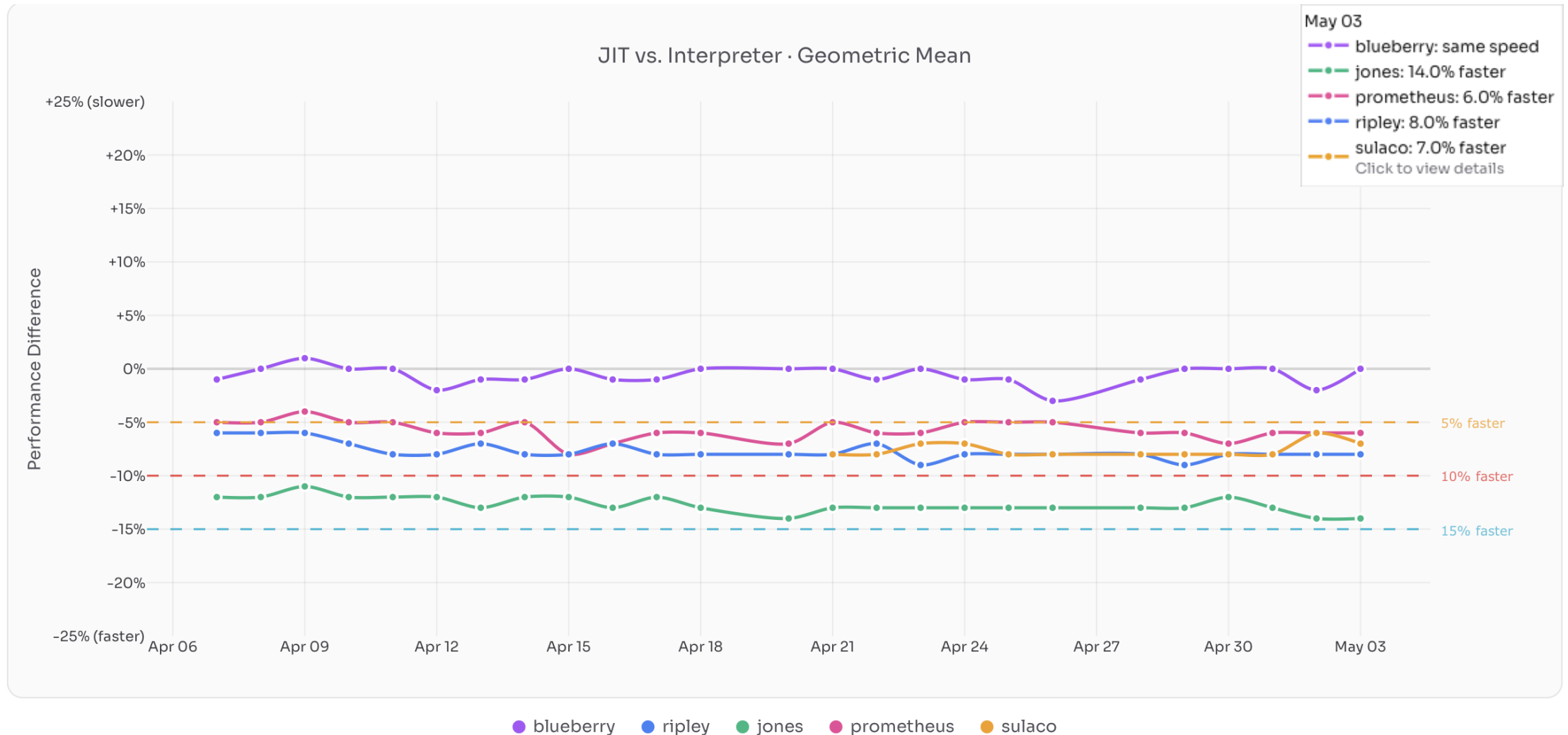
■ Baseline ■ Default ■ JIT



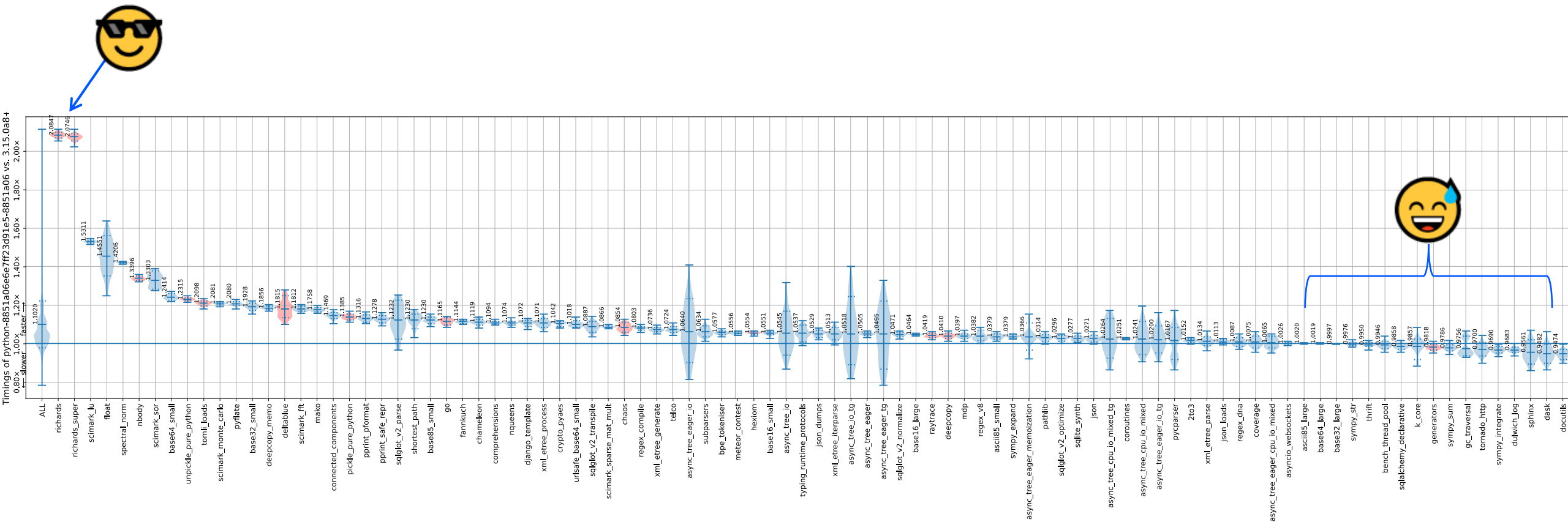
Bar heights use the geometric mean of per-benchmark mean runtimes on the global common set.

What does the JIT actually give you?

<https://www.doesjitgobrrr.com/>

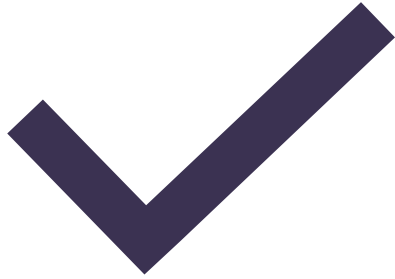


JIT impact across workloads



What to expect in 3.15

What users should expect in 3.15

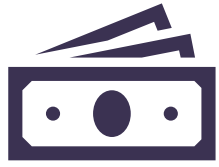


A more practical JIT



Benchmark your own workload

What distributors and tool authors should expect



Build cost



Platform support

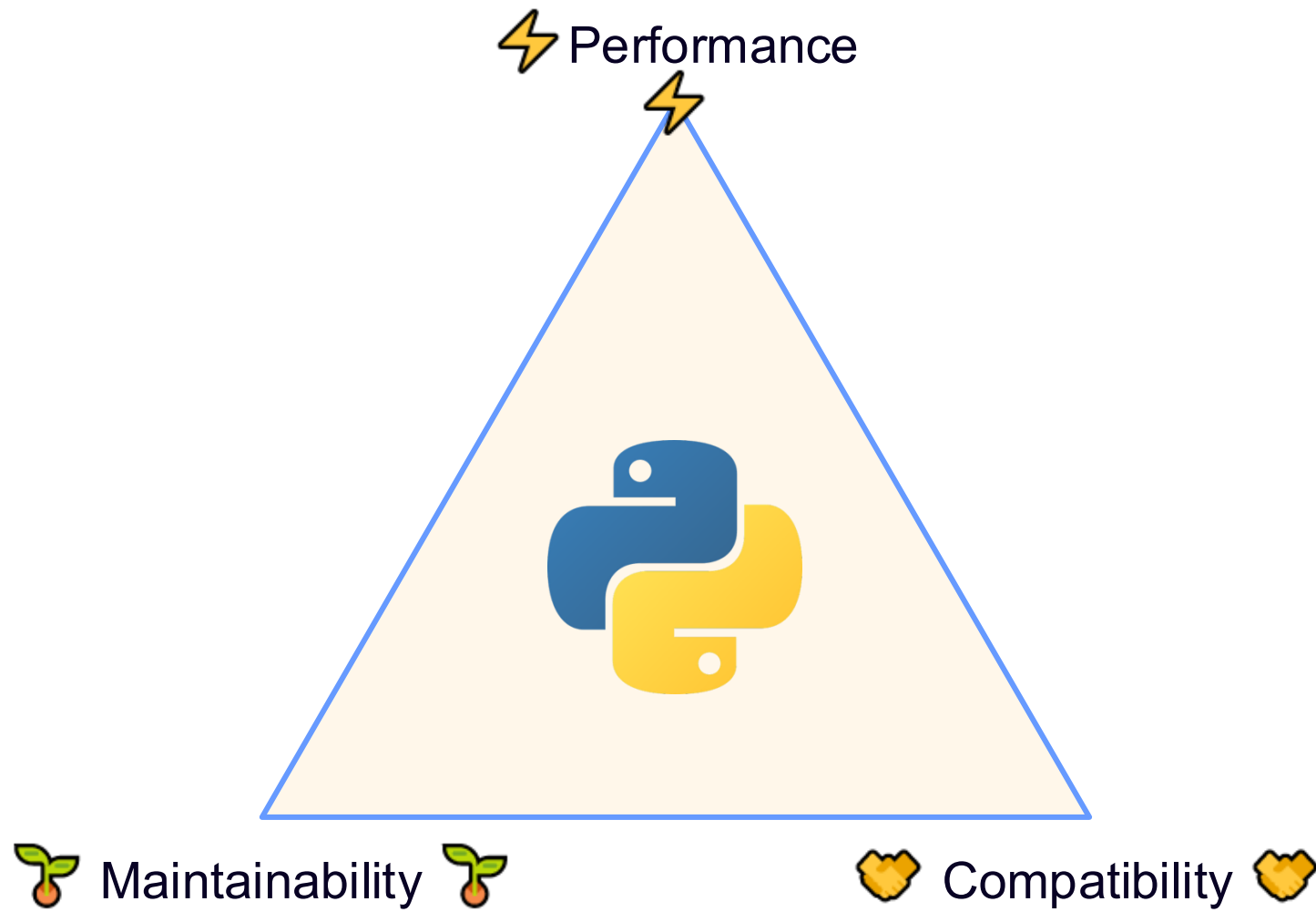


Debugging



Maintenance load

Beyond 3.15



From experiment to commitment 

A proposed way forward 

The JIT is now a CPython design decision 

arm

Tack

ధన్యవాదగళు

Merci

Danke

Gracias

Grazie

谢谢

ありがとう

Asante

Thank you

감사합니다

ధన్యవాద

Kiitos

شكرًا

ধন্যবাদ

תודה

ధన్యవాదములు

Köszönöm